

UrbanSense_Master_Specification_v3.1

PART A — UNDERSTAND URBANSENSE

Executive Summary & Problem → Solution Public transport buses traverse nearly every city road daily, yet authorities rely on slow, manual inspections and citizen complaints. UrbanSense transforms existing bus fleets into edge-AI mobile sensors. The system detects 29 authoritative Problem Statement requirements spanning perception, infrastructure intelligence, traffic analytics, safety, mobility analytics, alerting, GIS, authority workflows, and edge efficiency. Instead of streaming raw video, buses transmit lightweight, quality-scored evidence. A central backend fuses this multi-bus evidence into a persistent, 8-state "RoadTwin" that tracks conditions from initial detection through to verified repair.

What UrbanSense Can / Cannot Know & Implementation Status

- **[AVAILABLE / IMPLEMENTED]** Edge CV detections, HAL, GPS/IMU traces, 8-state RoadTwin, Evidence Fusion, Negative Evidence, GIS mapping.
- **[PROPOSED]** Specific configurable threshold values (e.g., exact priority formula weights).
- **[RESEARCH]** Predictive deterioration modeling.
- **[NOT AVAILABLE] Passenger-level Origin-Destination (OD) data.** Passenger-level OD data is explicitly unavailable and outside the system's available-data scope. UrbanSense does not claim to infer individual passenger trips or true passenger OD matrices.
- **[IMPLEMENTED PROXY]** We implement a **Fleet Mobility OD Proxy**, estimating traffic flow using only bus trajectory and route progression.

System Overview: End-to-End Pothole Example

1. **Bus A detects pothole** → Edge creates Observation.
2. **Quality evaluated** → Event sent to backend.
3. **Road segment identified** → RoadTwin = **Observed**.
4. **Validation passed** → RoadTwin = **Candidate**.
5. **Bus B independently detects** → Fusion Engine aggregates independent evidence.
6. **Confidence threshold met** → RoadTwin = **Confirmed**.
7. **Maintenance workflow** → Priority scored → **Maintenance Pending**.
8. **Authority reports repair** → Dashboard updated → **Repair Reported**.
9. **Verification triggered** → Fleet actively monitors → **Verification Pending**.
10. **Multiple healthy buses pass** → No pothole detected → Negative Evidence accumulates.
11. **Threshold met** → Defect closed → **Verified Repaired**.
12. **Months later, new detection** → Defect returns → **Reappeared** (creates new evidence chain).

PART B — WHAT MUST BE BUILT

R01–R29 Requirement Matrix & Traceability

The system implements exactly 29 authoritative Problem Statement requirements.

ID	Requirement	Implementati on Module	Edge/Central	AT
R01	Pothole detection	Road Defect AI	Edge → Central	AT-01
R02	Damaged road detection	Road Defect AI	Edge → Central	AT-02
R03	Missing road dividers	Infrastructure AI	Edge → Central	AT-03
R04	Missing zebra crossings	Infrastructure AI	Edge → Central	AT-04
R05	Damaged traffic signs	Infrastructure AI	Edge → Central	AT-05
R06	Missing traffic signs	Infrastructure AI	Edge → Central	AT-06
R07	Waterlogging detection	Road Defect AI	Edge → Central	AT-07
R08	Other road hazards	Road Defect AI	Edge → Central	AT-08
R09	Vehicle detection	Vehicle AI	Edge	AT-09
R10	Vehicle classification	Vehicle AI	Edge	AT-10
R11	Vehicle counting	Vehicle AI	Edge → Central	AT-11
R12	Traffic density estimation	Traffic AI	Central	AT-12

R13	Bottleneck detection	Traffic AI	Central	AT-13
R14	Route delay estimation	Traffic AI	Central	AT-14
R15	Fleet Mobility OD Proxy	OD Analytics	Central	AT-15
R16	Vulnerable pedestrian risk	VRU AI	Edge	AT-16
R17	School child crossing risk	VRU AI	Edge	AT-17
R18	Rash driving detection	Incident AI	Edge	AT-18
R19	Hit-and-run detection	Incident AI	Edge	AT-19
R20	Offending vehicle tracking	Tracking AI	Edge	AT-20
R21	ANPR plate extraction	ANPR/OCR	Edge	AT-21
R22	ANPR metadata (Score/GPS)	Event Engine	Edge → Central	AT-22
R23	Secure alerting	Transport/Alert ing	Edge → Central	AT-23
R24	Central fleet aggregation	Evidence Fusion	Central	AT-24
R25	GIS visualization	GIS	Central	AT-25
R26	Congestion heat maps	GIS + Traffic AI	Central	AT-26

R27	Infra deficiency analysis	Maintenance AI	Central	AT-27
R28	Authority insights/workflows	Copilot/Workflows	Central	AT-28
R29	Edge bandwidth minimization	HAL/Transport	Edge	AT-29

MVP vs Full PS

MVP is the first complete vertical slice proving the architecture. Full PS ensures complete 29-requirement coverage.

Capability	MVP	Full PS
Road Defects	Core YOLO detector	Complete defect classes & persistence filters
Infrastructure	Missing-sign vertical slice	Complete expected-asset reasoning
Traffic	Vehicle counting	Density + bottleneck + route delay
Mobility OD	Fleet OD Proxy	Expanded fleet mobility analytics
VRU / Risk	Core risk detection	Full trajectory-based risk analytics
Incidents	Simulated kinematic incident	Full incident rolling-buffer workflow
ANPR	Incident-triggered OCR	Complete multi-frame ANPR workflow
RoadTwin	Core 8-state lifecycle	Full lifecycle analytics & recurrence history

GIS	Core spatial layers	Complete 20-layer GIS map set
Maintenance	Basic prioritization	Full configurable authority workflow
Copilot	Deterministic SQL queries	Full authority NLP assistant
Edge	Laptop / HAL abstraction	Jetson deployment & dynamic scheduling

PART C — HOW IT WORKS

System Architecture Overview

- **Edge Pipeline:** Camera/GPS/IMU → HAL → Inference → Object Tracking → Quality/Uncertainty Evaluation → Canonical Event Generation → MQTT Transport.
- **Central Pipeline:** Secure Ingestion → Deduplication → Spatial Map Matching → Evidence Fusion Engine → 8-State RoadTwin (PostGIS) → Analytics → GIS / Copilot.

Evidence Model: Health & Quality

Sensor health explicitly dictates evidence quality.

Sensor Health (Camera Status, GPS Accuracy) → Observation Quality (Visibility, Blur) →

Evidence Weight → RoadTwin Confidence.

Evidence Fusion Algorithm (MVP)

1. **Spatial Association:** Map-match coordinates to road_segment_id. Match entity using segment, geographic distance, time window, and object class.
2. **Evidence Weight:** Deterministic score isolating AI from physical reality.
 - $\text{evidence_weight} = \text{detector_confidence} \times \text{observation_quality} \times \text{gps_quality}$
3. **Independence:** Reject duplicate frames from the same bus pass. Require unique bus counts and temporal separation.
4. **Aggregate Evidence:** Configurable deterministic calculation.
 - $\text{aggregate_confidence} = 1 - \prod(1 - \text{evidence_weight}_i)$
5. **Lifecycle Decision:** Evaluate state transitions based on configurable thresholds.

Fusion Pseudocode:

```
Python
def on_observation(event):
    validate(event)
```

```

segment = map_match(event.location)
entity = associate(event, segment)

if not entity.exists():
    entity = create_entity(event)

weight = calculate_evidence_weight(event)

if is_independent(event, entity):
    add_positive_evidence(entity, event, weight)

update_aggregate_confidence(entity)
evaluate_lifecycle_transition(entity)
persist(entity)

```

Negative Evidence Algorithm

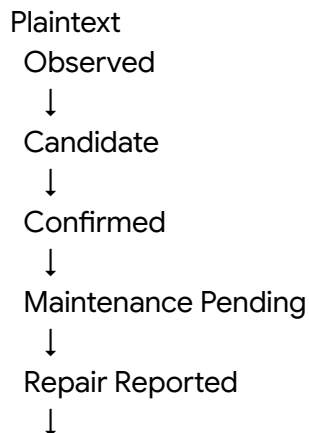
A missing detection is informative ONLY if all physical conditions allow for a detection.

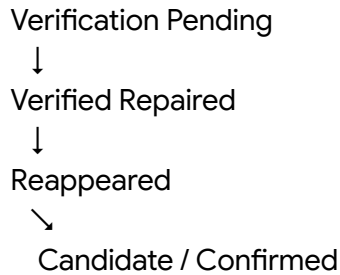
Formula: $\$NegativeEvidence = Coverage \times Visibility \times SensorHealth \times DetectionReliability\$$

Gate Logic:

1. Expected road location → Did the bus pass? (If NO → Discard)
2. Inside camera FOV? (If NO → Discard)
3. Visibility adequate? (If NO → Discard)
4. Occlusion acceptable? (If NO → Discard)
5. Sensor healthy? (If NO → Discard)
6. No defect detected → **Add Negative Evidence.**

8-State RoadTwin Lifecycle & Transitions





- **Observed → Candidate:** Triggered when initial detection passes validation and quality > threshold.
- **Candidate → Confirmed:** Triggered by independent corroboration (unique_bus_count ≥ 2 AND aggregate_confidence ≥ threshold). Action: enter maintenance queue.
- **Confirmed → Maintenance Pending:** Triggered by authority priority scoring.
- **Maintenance Pending → Repair Reported:** Triggered by manual authority input via UI. Action: flag for verification.
- **Repair Reported → Verification Pending:** Automatic transition. Trigger fleet to prioritize negative evidence collection.
- **Verification Pending → Verified Repaired:** Triggered when cumulative Negative Evidence meets strict repair-verification thresholds. Action: close record.
 - **Repair Verification Thresholds:**
 - min_independent_passes ≥ 3
 - min_coverage_per_pass ≥ 80%
 - sensor_health == OK
 - negative_evidence_score ≥ 0.85
 - new_positive_evidence == 0 over a 72-hour window.
- **Verified Repaired → Reappeared:** Triggered by new high-quality positive evidence at the exact site. Action: retain history, return to active defect state.

PART D — CAPABILITY CONTRACTS

Group 1: Road & Infrastructure Intelligence (R01–R08)

Requirements Covered: R01–R08

- **Input:** Front/Side Camera frames, GPS coordinates.
- **Data Source:** Edge HAL / Simulated feeds, GIS Expected Asset DB.
- **Processing:** YOLO detection + segmentation. For missing infrastructure, compare GIS Expected Assets vs. Observed Reality gated by Negative Evidence rules.
- **Output:** Canonical JSON event (event_type: pothole | missing_sign, confidence, quality).
- **Storage:** ROAD_DEFECT, INFRA_ASSET.
- **API/Event:** POST /events, GET /road-defects.
- **GIS/UI:** Plotted as vector layers on Road Health / Infra maps.
- **Failure/Fallback:** Low-quality images are discarded; sensor failure blocks Negative Evidence creation.
- **MVP:** Pothole and missing-sign vertical slice.
- **Full PS:** Complete 8-class defect/hazard coverage.

- **Definition of Done:** CV models integrated; output schema validated; API ingestion persists to DB; GIS renders points; test scenarios pass.

Group 2: Traffic & Fleet Mobility OD Proxy (R09–R15)

Requirements Covered: R09–R15

- **Input:** Multi-camera vehicle bounding boxes, Bus GPS Trajectories, Timestamps.
- **Data Source:** Edge CV, Bus GNSS/IMU.
- **Processing (R12–R14 Traffic):**
 - **R12 (Density):** Aggregate vehicle counts (R11) per segment divided by segment length over a time window.
 - **R13 (Bottleneck):** Flag segment if density exceeds high-threshold AND average fleet speed drops below low-threshold for a rolling 3-window period.
 - **R14 (Route Delay):** Compare observed bus segment traversal time versus scheduled/historical baseline travel time.
- **Processing (R15 OD Proxy):**
 1. Clean invalid GPS points.
 2. Map-match bus trajectories to route segments & zones.
 3. Identify Origin Zone → Destination Zone transitions.
 4. Aggregate by time window and normalize by coverage.
 5. Formula: $OD_{\{proxy\}}(i,j,t) = \frac{\sum \text{weighted fleet trajectories moving from zone } i \text{ to zone } j \text{ during time window } t}{\text{coverage}}$
- **Output:** traffic_observation, route_travel_time records, OD Proxy matrix.
- **Storage:** TRAFFIC_OBS, ROUTE_TIME.
- **API/Event:** GET /traffic, GET /od-proxy.
- **GIS/UI:** Congestion heatmaps, OD flow diagrams.
- **Definition of Done:** MAE counting error < threshold; R12-R14 logic triggers bottlenecks accurately; OD Proxy algorithm executed and visualized; Passenger OD disclaimer explicitly present in UI.

Group 3: Safety, Incidents & ANPR (R16–R22)

Requirements Covered: R16–R22

- **Input:** Camera frames, Trajectories, IMU data, Geofences.
- **Data Source:** Edge sensors, GIS School Zones.
- **Processing (R16–R20 Incidents):**
 - **R20 (Tracking):** Multi-Object Tracking (MOT) linking vehicle bounding boxes across consecutive frames using Kalman filters/IOU to assign persistent ID.
 - **R16 (VRU Risk):** Calculate pedestrian trajectory vector vs vehicle trajectory vector. If intersection Time-To-Collision (TTC) < threshold, flag alert.
 - **R17 (School Zone):** If GPS is inside GIS school geofence, multiply VRU risk score by 1.5 and lower the TTC threshold.
 - **R18 (Rash Driving):** Triggered if IMU lateral/longitudinal acceleration > threshold OR via erratic lane-change CV tracking.
 - **R19 (Hit-and-Run):** Kinematic collision signature (sudden decel + impact) followed by offending vehicle (MOT) leaving the frame.

- **Processing (R21-R22 ANPR):** Triggered *only* on incident. Crop → OCR. Maintain three separate scores: Vehicle Detection, Plate Localization, OCR Confidence.
- **Output:** incident, anpr events with cropped evidence buffers (pre/post-event).
- **Storage:** INCIDENT, VEHICLE_TRACK, PLATE.
- **API/Event:** POST /events (P0 Priority). GET /incidents (RBAC restricted).
- **GIS/UI:** Incident overlay; restricted ANPR view.
- **Definition of Done:** Kinematic triggers tested; 3 separate confidences (Vehicle, Plate, OCR) persisted; P0 alert routes instantly; restricted API functions.

Group 4: Platform, GIS, Authority & Edge Efficiency (R23–R29)

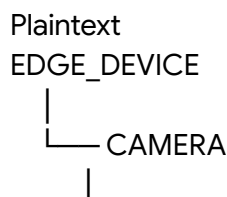
Requirements Covered: R23–R29

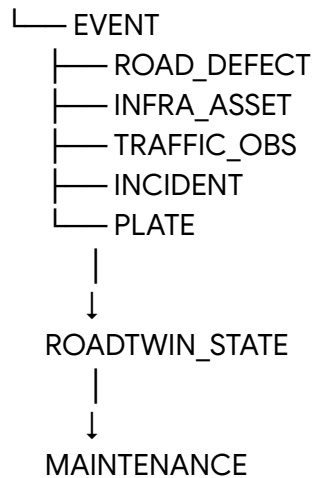
- **Input:** Edge events, RoadTwin state, Authority Prompts.
- **Data Source:** Edge MQTT, PostGIS DB, Web UI.
- **Processing:**
 - **R23 (Alerting):** RBAC-secured routing, automatic retry, audit logging based on P0-P3 event priority.
 - **R24 (Aggregation):** Centralized deduplication and spatial Evidence Fusion engine.
 - **R25 & R26 (GIS & Heatmaps):** Spatially aggregate TRAFFIC_OBS density scores into color-coded temporal heatmaps via MapLibre/Leaflet.
 - **R27 (Infra Deficiency):** Compare observed asset evidence vs GIS Expected Asset DB. Apply Negative Evidence gate. If expected asset is missing across min_independent_passes, increase deficiency_score.
 - **R28 (Insights):** NLP intent parsing → deterministic SQL tool call → Copilot explanation.
 - **R29 (Bandwidth):** Event classification categorizes payloads. P0 (Instant cellular) vs P3 (Buffered batch sync). Send small evidence crops, not continuous raw video.
- **Output:** Alerts, Map Layers, Copilot Text, Prioritized Bandwidth Queues.
- **Storage:** ALERT, MAINTENANCE, ROADTWIN_STATE.
- **API/Event:** GET /alerts, GET /gis/layers, POST /copilot/query.
- **Definition of Done:** Offline queueing tested; all 20 GIS layers render; R27 correctly flags missing signs; Copilot answers using strictly DB data.

PART E — ENGINEERING CONTRACTS

Database (PostgreSQL / PostGIS)

Compact ER structure mapping core relationships:





Key Fields per Entity:

- **EVENT:** event_id, bus_id, timestamp, location, road_segment_id, event_type, confidence, quality, evidence_ref.
- **ROADTWIN_STATE:** roadtwin_id, road_segment_id, object_type, lifecycle_state, aggregate_confidence, positive_evidence_count, negative_evidence_count, unique_bus_count, first_seen, last_seen, repair_status.
- **MAINTENANCE:** maintenance_id, roadtwin_id, priority, reported_at, repair_reported_at, verification_status.

APIs and Events

Endpoint	Method	Purpose / Payload
/events	POST	Ingest standardized evidence event
/roadtwin/{id}	GET	Retrieve current 8-state RoadTwin status
/road-defects	GET	Query defects for GIS and dashboards
/traffic	GET	Traffic density and route delay analytics
/od-proxy	GET	Fleet mobility OD Proxy matrix data
/incidents	GET	Incident evidence and

		ANPR (RBAC Restricted)
/maintenance/{id}/repair	POST	Authority triggers Repair Reported state
/gis/layers	GET	Fetch GeoJSON layer data for visualization
/alerts	GET	Authority PO/P1 alerts

HAL & Sensor Health Integration

- **Hardware Abstraction Layer (HAL):** Abstraction for Camera, GNSS, and Inference, allowing seamless migration from dev laptop to Jetson.
- **Sensor Health:** Continually tracks camera_availability, frame_rate, and gps_accuracy. Low sensor health drastically reduces observation_quality, preventing unhealthy edge nodes from generating false Negative Evidence.

PART F — VALIDATION

Requirement → Test Traceability & Acceptance Tests

Every R01–R29 requirement maps to a specific System Acceptance Test (AT).

ID	Input	Expected Result / Output
AT-01 to AT-08 (Defects)	Video of physical defect / hazard.	Edge detects object, creates event, backend generates Candidate.
AT-09 to AT-10 (Vehicles)	Video of mixed Indian traffic.	Detects bounding boxes, correctly classifies car/bus/truck/2W.
AT-11 to AT-12 (Density)	Video of dense traffic flow.	Count accuracy within 15% of manual baseline; density metric stored.
AT-13 (Bottleneck)	GPS speed drops while density spikes over 3 windows.	Segment flagged as Bottleneck in ROUTE_TIME and GIS.

AT-14 (Delay)	Trajectory traversal is 20% slower than baseline.	Route Delay alert generated.
AT-15 (OD Proxy)	Multi-bus trajectory through Zones A → B.	OD flow matrix calculates transition counts normalized by coverage.
AT-16 to AT-17 (VRU)	Pedestrian trajectory intersects vehicle in school zone.	High-priority PO Alert triggered. Risk score multiplied by 1.5.
AT-18 to AT-19 (Incident)	IMU anomaly / kinematic crash signature.	Rolling pre/post video buffer captured and sent to central.
AT-20 (Tracking)	Vehicle passes behind light post.	MOT assigns same ID before and after occlusion.
AT-21 to AT-22 (ANPR)	Incident triggers plate capture.	3 distinct confidences logged; GPS/Time appended to PLATE record.
AT-23 (Alerting)	P0 Event vs P3 Event generated.	P0 sent immediately via MQTT; P3 batched for later sync.
AT-24 (Fusion)	Two identical events from same bus pass.	Deduplication engine rejects duplicate; requires independent bus.
AT-25 to AT-26 (GIS)	PostGIS populated with test events.	All 20 GIS layers render correctly; Heatmap visualizes density.
AT-27 (Infra Def.)	Clear camera pass over missing sign location.	Negative Evidence triggers deficiency_score increase.
AT-28 (Copilot)	Prompt: "Show recurrent potholes."	Copilot executes SQL tool call; returns strictly

		DB-grounded data.
AT-29 (Edge)	Network connection drops.	Edge buffers events offline and successfully syncs upon reconnection.

CRITICAL AT-LIFECYCLE: Input a simulated timeline to verify the exact progression:

Observed → Candidate → Confirmed → Maintenance Pending → Repair Reported →

Verification Pending → Verified Repaired → Reappeared.

Test MUST not skip states and MUST verify the Reappeared recurrence loop.

Definition of Done (DoD) Standard

A module is strictly done when: Implementation complete → Input validated → Output schema

matches canonical event → API integrated → PostGIS persistence working → GIS/UI updated

→ Failure/fallback handled → Test cases & Demo scenarios pass.

PART G — RESEARCH + PRESENTATION

Established vs. Engineering vs. Research

- **Established Technology:** YOLO detection, OCR, GPS, PostGIS, MQTT, ONNX. (Do not claim these as novel).
- **Engineering Contribution:** Bus fleet sensing architecture, HAL portability, Canonical Event integration, 20-layer GIS visualization, Copilot authority workflows.
- **Research Contribution:** Lifecycle-aware evidence fusion, Negative Evidence math for repair verification, coverage-aware confidence, reappearance tracking.
 - *Research Format:* Problem (Manual repair confirmation) → Method (Negative Evidence Gating) → Input (Camera FOV/Quality) → Output (Verified Repaired state) → Evaluation (Ablation study against raw single-shot detection).

PPT Storyline & Demo Flow

1. **The Gap:** Authorities lack real-time data; fixed CCTV is blind; manual audits are slow.
2. **The Solution:** 29 authoritative Problem Statement requirements fulfilled via edge-AI bus cameras.
3. **The Innovation:** We move beyond raw detections. Our architecture uses Evidence Fusion and Negative Evidence to build an 8-state RoadTwin.
4. **Feasibility & Demo:**
 - *Beat 1:* Bus A detects pothole (Observed).
 - *Beat 2:* Bus B corroborates (Confirmed).
 - *Beat 3:* UI marks repair (Verification Pending).
 - *Beat 4:* Subsequent healthy buses confirm absence via Negative Evidence (Verified Repaired).
 - *Beat 5:* Show Copilot querying the deterministic PostGIS backend for insights.